Kernel PCA

2026-04-17

Introduction

Kernel principal component analysis performs PCA in a high-dimensional feature space implicitly defined by a kernel function, capturing non-linear structure that linear PCA misses. The classic example — two concentric circles — illustrates the point: linear PCA fails to separate them because no straight axis distinguishes the two classes, but kernel PCA with a radial-basis-function kernel finds the radial coordinate in an implicit infinite-dimensional space and separates the rings cleanly. The technique generalises PCA to data lying on curved manifolds while preserving its eigen-decomposition mathematical structure.

Prerequisites

A working understanding of principal components analysis, the kernel trick, Gram matrices, and the difference between linear and non-linear feature spaces.

Theory

A kernel function $K(x, y) = \langle \phi(x), \phi(y) \rangle$ defines an inner product in a feature space without ever computing ϕ explicitly (the "kernel trick"). Standard choices:

- **RBF / Gaussian**: $K(x, y) = \exp(-\gamma \|x - y\|^2)$, infinite-dimensional feature space.
- **Polynomial**: $K(x, y) = (x^\top y + c)^d$, polynomial feature space of degree d.
- **Sigmoid**: $K(x, y) = \tanh(\alpha x^\top y + c)$, related to neural-net activations.

Construct the $n \times n$ Gram matrix K, centre it in feature space, and compute its top eigenvectors. The projection of an observation onto a kernel principal component is a weighted sum of kernel evaluations against all training observations — making kernel PCA $O(n^2)$ in space and time, which becomes prohibitive for $n \gg 10{,}000$.

Assumptions

The kernel encodes a similarity meaningful for the inferential question; the Gram matrix is positive semi-definite (true for valid Mercer kernels); sufficient observations for stable eigendecomposition.

R Implementation

```
library(kernlab)

# Data on two concentric circles (linear PCA fails)
set.seed(2026)
theta <- runif(200, 0, 2 * pi)
r <- c(rep(1, 100), rep(3, 100))
X <- cbind(r * cos(theta), r * sin(theta)) + matrix(rnorm(400, 0, 0.1), 200)
class <- factor(r)

# Linear PCA
pca <- prcomp(X)
```

```r
plot(pca$x[, 1:2], col = class, main = "Linear PCA")

# Kernel PCA with RBF
kpca <- kpca(X, kernel = "rbfdot", kpar = list(sigma = 0.1), features = 2)
plot(rotated(kpca), col = class, main = "Kernel PCA (RBF)")
```

Output & Results

Two side-by-side scatterplots: linear PCA leaves the two concentric rings overlapping in PC space, while kernel PCA with RBF kernel and $\sigma = 0.1$ separates them cleanly along the first kernel principal component.

Interpretation

A reporting sentence: "Kernel PCA with an RBF kernel ($\sigma = 0.1$) separated the two concentric rings (n = 100 each) that linear PCA could not, demonstrating that the underlying class structure is non-linear in the original feature space; the first kernel principal component captured the radial coordinate that distinguishes the classes." When kernel PCA succeeds where linear PCA fails, name the geometric feature being captured.

Practical Tips

- Kernel choice and hyperparameters matter; RBF with median-heuristic bandwidth is a reasonable default, then tune by cross-validation on a downstream supervised task.
- Kernel PCA components are not directly interpretable in the original feature space; they are weighted sums of kernel evaluations against training points.
- Computation scales as $O(n^2)$ space and $O(n^3)$ time for the eigendecomposition; for $n > 10{,}000$, use Nyström approximation (`kernlab::kha`, `RSpectra` for partial eigendecomposition).
- Modern alternatives for non-linear visualisation — UMAP, t-SNE, autoencoders — often outperform kernel PCA for cluster separation in two dimensions.
- Kernel PCA is most useful as a feature engineering step for downstream supervised learning when p is very small; in $p \gg n$ regimes, sparse linear methods (PLS, lasso) are typically more efficient.
- For preserving local geometry, prefer Laplacian eigenmaps or diffusion maps; kernel PCA preserves variance in the implicit feature space, not local distances.

R Packages Used

`kernlab` for `kpca()` and a wide range of kernels; `RSpectra` for partial eigendecomposition of large Gram matrices; `uwot` (UMAP) and `Rtsne` for modern non-linear embedding alternatives; `dimRed` for a unified interface across kernel PCA, t-SNE, UMAP, and Isomap.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- PCA, factor analysis, CCA, LDA
- Clustering: k-means, hierarchical, model-based
- UMAP and t-SNE